Jing Cheng

Final Project in Computational Literacy

Themes in song lyrics across decades

Introduction

When trying to choose a topic for this final project and aligning with my personal interest. I've always been fascinated by music literature, the way that written lyrics can convey emotions and represent current societal issues. So I wanted to choose a topic that focuses on the analysis of songs. Music is a multisensory system experiences, but considering the difficulty of analyzing visual music video or auditory melody, I've decided to focus on song lyrics for my final project. The goal is to identify the recurring themes and evaluate for differences or similarity in the number one hit song across decades. Though I am doing my bachelor's in computer science, I will admit that coding is not my strongest suit and therefore did not want to overcomplicate it. So my idea is to use natural language processing (NLP) techniques such as topic modeling(LDA) and semantic interpretation(zero-shot classifier) to identify the themes and analyze the lyrics.

- Research Questions

1. *"What recurring themes dominate hit songs from different decades"*
2. *"How do different method compare in identifying themes?"*

Data Collection and preprocessing

To narrow the scope of analysis, I chose the hit songs from 1970s, 80s and 90s respectively. Due to problems such as lack of available archived data and charts from Billboard, as I originally plan to extract data from that. I decided chose five widely acclaimed songs from each decades given some research from this Kaggle source, I've selected 5 different artist in each time frame and proceed to selected their biggest hit song for analysis.

:

1970s

1. *"Stairways to Heaven" by Led Zeppelin (1971)*
2. *"Rocket man" by Elton John (1972)*
3. *"Us and Them" by Pink Floyd (1973)*
4. "*Bohemian Rhapsody" by Queen (1975)*
5. *"Dreams" by Fleetwood Mac (1977)*

1980s

1. *"Thriller" by Michael Jackson (1982)*
2. *"Purple Rain" by Prince (1984)*
3. *"Like a Virgin" by Madonna (1984)*
4. *"Running Up That Hill" by Kate Bush (1985)*
5. *"With or Without You" by U2 (1987)*

1990s

1. *"End of the Road" by Boyz II Men (1992)*
2. *"I Will Always Love You" by Whitney Houston (1992)*
3. *"That's the Way Love Goes" by Janet Jackson (1993)*
4. *"All I Want for Christmas Is You" by Mariah Carey (1994)*
5. *"No Scrubs" by TLC (1999)*

The next step, I extracted the lyrics from Genius, an open-source platform, and saved them in txt. file. Given the variety in time period, artist, song genre and especially popularity, I would consider them representative of their respective decades in terms of impact, as these are all hit songs that the general public can agree on.

Then proceeding to preprocessing of the raw data, I've taken measures to remove common filler and stopwords from the lyrics, lowercase the text and remove punctuations and special characters. This is done through the help of code provided from GeeksforGeeks. I then added some custom and more refined stopwords myself, such as "ohh, woo, mm", that are commonly presented in lyrics. In which does not add much content to the meaning of lyrics.
In the code shown below, we first import stopword list from *nltk* library, and after creating some custom stopwords, I join the stopwords together in one set. Then proceed to breakdown texts into individual words. Then filtering out the words from stopword list, we get a list of words in "x" format. Next step is to lowercase these words and then to remove punctuations. Last step is to join the word into one string (i.e., text format again).

```
import nltk
nltk.download()

from nltk.corpus import stopwords
nltk.download('stopwords')

nltkstopwords = set(stopwords.words('english'))

extraword = nltkstopwords.union({'oh','ooh','mm','hmm','ah','wooo','gon','na'})

print(extraword)

from nltk.tokenize import word_tokenize

lyrics = """She packed my bags last night, pre-flight
Zero hour: 9:00 AM
And I think it's gonna be a long, long time"""


tokens = word_tokenize(lyrics)

filtered = [w for w in tokens if not w.lower() in extraword]

filtered = []

for w in tokens:
    if w not in extraword:
        filtered.append(w)

print(tokens)
print(filtered)

import string

filtered = [w.lower() for w in tokens if w.isalpha() and w.lower() not in extraword]

print("Filtered Words:")
print(filtered)

cleaned_text = " ".join(filtered)

print("Cleaned text:")
print(cleaned_text)
```

(The code showcases the example of when cleaning the lyrics for “rocket man”. Part of the lyrics has been cropped out when screenshotting the VS code page)

The processed and cleaned text were copied from terminal to a folder all under txt.file. This allows us to proceed to the next step—identifying the themes in the processed texts respectively.

Zero-shot Classifier

Firstly, we manually go through 2 song lyrics from each decade and select relevant theme respectively. From the following list common themes presented in lyrics/literature work: “Love”, “Freedom”, “Dreams”, “Faith”, “Existential”, “Loneliness”

Then using semantic interpreter tool to capture themes computationally for comparison. I decided to utilize a pre-trained model from Hugging Face transformers. I searched in Hugging face for natural language processing model, and decided to employ a zero-shot classification tool called facebook/bart-large-mnli. As zero shot classification is when *a model is trained on a set of labeled examples but is then able to classify new examples from previously unseen classes*(https://huggingface.co/tasks/zero-shot-classification). This specific model chosen can essentially classify text into pre-defined themes without requiring additional training.

```python
import os
from transformers import pipeline

classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli", device=0)

single_file = "/Users/jcmac/Downloads/lyrics-analysis/cleaned-lyrics/70s/dreams.txt"

with open(single_file, 'r') as file:
    lyrics = file.read()

result = classifier(lyrics, ["Love", "Freedom", "Dreams", "Faith", "Existential", "Loneliness"], multi_label=True)

print(os.path.basename(single_file))
for label, score in zip(result["labels"], result["scores"]):
    print(f"Theme: {label}, Confidence: {score:.2f}")

```

We run the song individually through the classifier and see what themes it outputs.
For the 70s:

Song	Themes (manually selected)	Themes (classifier)
"Dreams" by Fleetwood Mac	**Freedom, Dreams, Loneliness**	**Loneliness, Freedom, Dreams**
"Stairways to Heaven" by Led Zeppelin	Existential (0.56), **Faith**	**Faith**, Dreams (0.79)

```
dreams.txt                                stairway to heaven cleaned.txt
Theme: Loneliness, Confidence: 0.99       Theme: Faith, Confidence: 0.83
Theme: Freedom, Confidence: 0.99          Theme: Dreams, Confidence: 0.79
Theme: Dreams, Confidence: 0.98           Theme: Love, Confidence: 0.68
Theme: Faith, Confidence: 0.78            Theme: Existential, Confidence: 0.56
Theme: Love, Confidence: 0.75             Theme: Freedom, Confidence: 0.51
Theme: Existential, Confidence: 0.57      Theme: Loneliness, Confidence: 0.06
```

(Selecting top 2 or top 3 themes from classifier depending on the number of manual themes selected)

80s:

Song	Themes (manually selected)	Themes (classifier)
"Purple Rain" by Prince	**Love**, Freedom (0.41)	**Love**, Dreams (0.70)
"Running Up That Hill" by Kate Bush	**Faith, Loneliness**, Dreams (0.60)	**Loneliness, Faith,** Love (0.77)

```
purple rain cleaned.txt
Theme: Love, Confidence: 0.76
Theme: Dreams, Confidence: 0.70
Theme: Faith, Confidence: 0.65
Theme: Loneliness, Confidence: 0.49
Theme: Freedom, Confidence: 0.41
Theme: Existential, Confidence: 0.36
```

```
running up that hill cleaned.txt
Theme: Loneliness, Confidence: 0.88
Theme: Faith, Confidence: 0.78
Theme: Love, Confidence: 0.77
Theme: Freedom, Confidence: 0.67
Theme: Dreams, Confidence: 0.60
Theme: Existential, Confidence: 0.41
```

90s:

Song	Themes (manually selected)	Themes (classifier)
"End of the Road" by Boyz II Men	**Love, Loneliness**	**Love, Loneliness**
"I Will Always Love You" by Whitney Houston	**Love**, Loneliness (0.07)	**Love**, Dreams (0.90)

```
end of the road cleaned.txt
Theme: Love, Confidence: 0.99
Theme: Loneliness, Confidence: 0.95
Theme: Faith, Confidence: 0.83
Theme: Dreams, Confidence: 0.75
Theme: Existential, Confidence: 0.44
Theme: Freedom, Confidence: 0.43
```

```
i will always love you cleaned.txt
Theme: Love, Confidence: 0.98
Theme: Dreams, Confidence: 0.90
Theme: Faith, Confidence: 0.67
Theme: Freedom, Confidence: 0.62
Theme: Existential, Confidence: 0.54
Theme: Loneliness, Confidence: 0.07
```

From this comparison, we can observe significant overlaps between the manual prediction of themes manually and computational prediction through the classifier model (as indicted in red bold font in the table). This suggests that the model is effective in identifying the core conceptual themes, such as Love and Loneliness, which are presented across multiple songs.

However, differences do appear at times as well, in which I've indicated the difference in confidence level that classifier produced. The numerical difference across songs is not of significance except for lyrics of "I will always love you" between Loneliness(manual) and Dreams(classifier). This difference of 0.07 and 0.90 could potentially be due to multiple reasons. For example, ambiguity in interpretation. When manually extracting the theme, we tend to consider the song as a whole. And human interpretation can be biased in factors such as melody, instrumentation choice of song, prior knowledge of the artist's intent or history, and sometimes even abstract interpretation such as the vibe of a song, whilst the model focuses purely on textual lyrics, especially the pre-processed version as well. I've considered inputting the raw lyrics data without the removal of stopword into this. However, I think it might be difficult to analyze those text, given the cluster of irrelevant words such as "you", "I". That can potentially cause model to give unprecise prediction, for example, labelling it heavily as theme Freedom and Existential.

Another reason could be due to the vagueness of theme definition, for example the theme Dreams, have multiple meanings. Including one's aspiration or the subconscious stage we entered when sleeping, which then the classifier could interpret differently in comparison to a human's perspective. This difference in interpretation may even arise between individuals, nevertheless a computational model.

LDA Model

The next step, we employ Latent Dirichlet Allocation (LDA) tool. This method allow us to analyze for the theme without pre-defining, but purely based on distribution of textual data. Where we compile and feed all cleaned lyrics across all decades to the model at the same time. Then we assign 6 topics to align with the pre-defined 6 themes from the zero-shot classifier modal.

```python
import os
import numpy as np
import pandas as pd
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.preprocessing import normalize

path = "/Users/jcmac/Downloads/lyrics-analysis/cleaned-lyrics"
alllyrics = []
titles = []

for decade in os.listdir(path):
    dpath = os.path.join(path, decade)
    if os.path.isdir(dpath):
        for filename in os.listdir(dpath):
            if filename.endswith(".txt"):
                with open(os.path.join(dpath, filename), 'r', encoding='utf-8') as file:
                    alllyrics.append(file.read())
                    titles.append(filename)

vectorizer = CountVectorizer(max_df=0.95, min_df=2)
x = vectorizer.fit_transform(alllyrics)
topics = 6
ldamodel = LatentDirichletAllocation(n_components=topics, random_state=42)
ldamodel.fit(x)

def display(model, features, number_words=1):
    for topic_idx, topic in enumerate(model.components_):
        top_words = [features[i] for i in topic.argsort()[-number_words:]]
        print(f"Topic {topic_idx + 1}: {', '.join(top_words)}")

print("Interpreted topics:")
display(ldamodel, vectorizer.get_feature_names_out())

distribution = ldamodel.transform(x)
distribution = normalize(distribution, norm='l1', axis=1)

topicsdf = pd.DataFrame(distribution, columns=[f'Topic {i+1}' for i in range(topics)], index=titles)

print(topicsdf)
```

The output is as follows:

```
Interpreted topics:
Topic 1: know
Topic 2: go
Topic 3: long
Topic 4: love
Topic 5: ca
Topic 6: could
```

	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
dreams.txt	0.984155	0.003160	0.003164	0.003192	0.003167	0.003162
stairway to heaven cleaned.txt	0.986494	0.002699	0.002706	0.002692	0.002703	0.002706
rocket man cleaned.txt	0.001390	0.001391	0.993048	0.001391	0.001391	0.001390
us and them cleaned.txt	0.002837	0.002850	0.084690	0.138837	0.002866	0.767920
bohemian rhapsody cleaned.txt	0.001453	0.992711	0.001457	0.001462	0.001460	0.001456
thats the way love goes cleaned.txt	0.001059	0.001061	0.001057	0.994699	0.001062	0.001060
no scrubs cleaned.txt	0.001258	0.001259	0.001262	0.001264	0.993698	0.001260
end of the road cleaned.txt	0.001155	0.001160	0.001154	0.001156	0.994219	0.001156
i will always love you cleaned.txt	0.003572	0.003588	0.003589	0.982112	0.003568	0.003571
all i want for christmas cleaned.txt	0.001879	0.001877	0.001878	0.001880	0.001880	0.990606
like a virgin cleaned.txt	0.002625	0.002618	0.631676	0.357835	0.002624	0.002623
purple rain cleaned.txt	0.002331	0.002337	0.988303	0.002327	0.002353	0.002348
without or without you cleaned.txt	0.003160	0.003158	0.003155	0.984176	0.003180	0.003171
thriller cleaned.txt	0.001447	0.001449	0.001446	0.992757	0.001453	0.001448
running up that hill cleaned.txt	0.002466	0.002466	0.002456	0.002476	0.002479	0.987658

```
jcmac@Js-MacBook-Air-2 ~ %
```

Now we obtain the topic and the loadings for each song, we can analyze whether they make sense compared to our manual tagging and classifier tagging. I assigned the model to produce both 1 word for each topic to better the analyze and align with the selected theme.
Some words could be said to loosely relate to the pre-defined themes. For example, topic 4(love) and “Love”, topic 2(go) and “Freedom”. However, there exist topics that are quite irrelevant to the chosen theme as well. For example, “ca”, which appears to not have a literal meaning to it. This could be caused by issues in preprocessing or vectorizing of the data. “could” “long” and “know” are also more or less just common words that fill up in a sentence.

For the purpose of the comparison, I altered the model and assign 3 words for each topic, in the attempt to gain more insights into the matching of each topic to the pre-defined theme as follows to analyze for each song.

```
Interpreted topics:
Topic 1: makes, go, know
Topic 2: really, let, go
Topic 3: time, rain, long
Topic 4: night, way, love
Topic 5: want, love, ca
Topic 6: make, want, could
```

	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
dreams.txt	0.984155	0.003160	0.003164	0.003192	0.003167	0.003162
stairway to heaven cleaned.txt	0.986494	0.002699	0.002706	0.002692	0.002703	0.002706
rocket man cleaned.txt	0.001390	0.001391	0.993048	0.001391	0.001391	0.001390
us and them cleaned.txt	0.002837	0.002850	0.084690	0.138837	0.002866	0.767920
bohemian rhapsody cleaned.txt	0.001453	0.992711	0.001457	0.001462	0.001460	0.001456
thats the way love goes cleaned.txt	0.001059	0.001061	0.001057	0.994699	0.001062	0.001060
no scrubs cleaned.txt	0.001258	0.001259	0.001262	0.001264	0.993698	0.001260
end of the road cleaned.txt	0.001155	0.001160	0.001154	0.001156	0.994219	0.001156
i will always love you cleaned.txt	0.003572	0.003588	0.003589	0.982112	0.003568	0.003571
all i want for christmas cleaned.txt	0.001879	0.001877	0.001878	0.001880	0.001880	0.990606
like a virgin cleaned.txt	0.002625	0.002618	0.631676	0.357835	0.002624	0.002623
purple rain cleaned.txt	0.002331	0.002337	0.988303	0.002327	0.002353	0.002348
without or without you cleaned.txt	0.003160	0.003158	0.003155	0.984176	0.003180	0.003171
thriller cleaned.txt	0.001447	0.001449	0.001446	0.992757	0.001453	0.001448
running up that hill cleaned.txt	0.002466	0.002466	0.002456	0.002476	0.002479	0.987658

“Love”,	“Freedom”	“Dreams”	“Faith”	“Existential”	“Loneliness”
Topic 4	Topic 2	Topic 6	Topic 5	Topic 1	Topic 3

Recalling the 2 chosen songs from each decade earlier. We choose the topic with >0.95 weights as the theme extracted from LDA.

Song	Themes (Manual)	Themes (LDA)
“Dreams” by Fleetwood Mac	Freedom, Dreams, Loneliness	Existential
“Stairways to Heaven” by Led Zeppelin	**Existential**, Faith	**Existential**

“Purple Rain” by Prince	Love, Freedom	Loneliness
“Running Up That Hill" by Kate Bush	Faith, Loneliness, **Dreams**	**Dreams**
“End of the Road” by Boyz II Men	Love, Loneliness	Faith
“I Will Always Love You” by Whitney Houston	**Love**, Loneliness	**Love**

We can observe that there are songs with overlapping themes like "Love", "Existential" and “Dream”. There are however themes that do not overlap with the manual prediction at all. Main reason could be that, the chosen method to assign topics from LDA model to pre-defined themes together is simple and intuitive. This match or rather mismatch does not guarantee for accuracy since it was fully up to my personal subjective interpretation. The presentation of potential human bias could lead to inconsistencies and inaccuracy as well. Additionally, though this method provided some insights, the topics that the model identified were still quite vague and generic, seemingly dominated by frequently repeated words in the lyrics rather than its actual conceptual meanings.

Discussion

By comparing the LDA model with manually selected themes and zero-shot classification results, pros and cons of both appeared. Due the nature of LDA model, it extracted the theme based on distribution of frequently presented words in the lyrics. This offers insight into the clear prominence on words, and assign each song individually to the produced topic, which display the relationship and relevance between each lyric and topic numerically. This allows for straightforward interpretation. However as mentioned before, it still fails to capture the conceptual themes and the results are rather inaccurate in identifying a deeper meaning into the lyrics, especially in comparison with the manual extraction of theme. In contrast, the hugging face transformer achieved the goal more, it is more effective in extracting actual conceptual themes and align better with the manual selection. However due to its reliance on pre-trained models, there is limitation in flexibility and transparency. Resulting in difficulty in for example, modifying and finetuning. As the method reply heavily on comparison analysis, between manual selected theme and theme identified through both models. Potential biases in manual tagging, that are due to individual emotional perception of songs could play an important in comparing the accuracy of said models as well.

Referring back to the other research question on emotions, I would say the classifier model is effective or at least able to reflect and identify themes in popular songs from the chosen centuries. Though this can be improved by adding more specific or wider range of pre-defined themes. The LDA are less effective in that aspect as discussed previously.

Some other limitations included the structural bias in song lyrics. The nature of song lyrics structure causes certain words to disproportionately dominate the results. For example, words used in chores as it repeated significantly more compared to other parts of lyrics, such as bridges. This affects the overall judgement and can negatively impact the results generated from the model as well. The limitation of a dataset, consisting of only five songs per decades restricts the generalizability of insights and learnability of models as well. This is partially due to my choice of pre-processing method, in which the original lyrics are cleaned manually one by one. Therefore, with a biggest dataset, it would be time consuming.

Improvements

A broader and more representative dataset, for example with more songs from each decade, could provide a better insight. And this could be implemented through the implementation of better pre-processing techniques to for example, automate data cleaning and therefore expand the dataset.

We can train and fine-tune the Hugging Face transformer or with the help of it, design a more relevant and accurate lyrics and theme extraction model. And further perfect the selecting of pre-defined themes to increase diversity so that a comparative analysis across the songs from each decade can be incorporated in the future.